

# Number System

09 June 2026 05:08

How are numeric data items actually stored in the computer memory?

How much space(memory locations) is allocated for each type of data?

- byte, short, int, long, float, double, char, boolean.

How are characters and strings are stored in memory?

## Number System:: The basics

We are accustomed to using the so called **decimal number system**.

- Ten digits:: 0,1,2,3,4,5,6,7,8,9
- Every digit position has a weight which is a power of 10.
- Base or radix is 10

Example:

$$234 = 2 * 10^2 + 3 * 10^1 + 4 * 10^0$$

$$250.67 = 2 * 10^2 + 5 * 10^1 + 0 * 10^0 + 6 * 10^{-1} + 7 * 10^{-2}$$

## Binary Number System

Two digits:

- 0 and 1
- Every digit position has a weight which is a power of 2.
- Base or radix is 2

Example:

$$110 = 1 * 2^2 + 1 * 2^1 + 0 * 2^0$$

$$110.01 = 1 * 2^2 + 1 * 2^1 + 0 * 2^0 + 0 * 2^{-1} + 1 * 2^{-2}$$

## Binary to decimal conversion

Each digit position of a binary number has a weight.

- Some power of two

A binary number:

$$B = b_{n-1}b_{n-2}...b_1b_0b_{-1}b_{-2}...b_{-m}$$

Corresponding value in decimal:

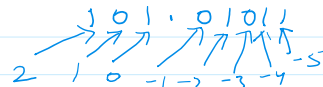
$$D = \sum_{i=-m}^{n-1} b_i \cdot 2^i$$

algorithm

$$\sum_{i=-5}^{3-1=2} b_i \cdot 2^i$$

n = number of digits in integer part

every digit is b



Examples:

$$\begin{aligned} 1010101_2 &= 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 \\ &= 32 + 0 + 8 + 0 + 2 + 1 \\ &= 43_{10} \end{aligned}$$

$$0.0101_2 = 0 \times 2^{-1} + 1 \times 2^{-2} + 0 \times 2^{-3} + 1 \times 2^{-4}$$

$$\begin{array}{r} 2^{10} \cdot 1.1 \\ \hline \end{array} = 5 + 1 \times 2^{-1} + 1 \times 2^{-2}$$

$$5 + 0.50 + 0.25 = 5.75_{10}$$

```

bs = '1101'
print(int(bs,2)) ← binary to int
13
print(oct(13)) ← int to octal
0o15
print(hex(13)) ← int to hex
0xd
print(bin(13)) ← int to binary
0b1101

```

## Java

```
public class Coverersion
```

```

{
    public static void convertIntegerToBinary(int v){
        System.out.print("\nBinary equivalent of " + v + " is: " +
        Integer.toBinaryString(v));
        System.out.print("\n?: "+Integer.toUnsignedString(110, 2)); ← int is converted to binary (1101110)2
        System.out.print("\n ??: "+Integer.toString(1101, 16));
    }
}

```

Methods available in Java for conversion:

```

String toBinaryString(int)
String toHexString(int)
String toOctalString(int)
String toString(int)
String toString(int, int)
long toUnsignedLong(int)
String toUnsignedString(int)
String toUnsignedString(int, int)

```

Integer.to

## Hexadecimal Number System

A compact way of representing binary numbers  
16 different symbols(radix = 16)

16 = 2<sup>4</sup> ← bits

|   |   |   |   |        |
|---|---|---|---|--------|
| 0 | 0 | 0 | 0 | 0      |
| 0 | 0 | 0 | 1 | 1      |
| 0 | 0 | 1 | 0 | 2      |
| 0 | 0 | 1 | 1 | 3      |
| 0 | 1 | 0 | 0 | 4      |
| 0 | 1 | 0 | 1 | 5      |
| 0 | 1 | 1 | 0 | 6      |
| 0 | 1 | 1 | 1 | 7      |
| 1 | 0 | 0 | 0 | 8      |
| 1 | 0 | 0 | 1 | 9      |
| 1 | 0 | 1 | 0 | A (10) |
| 1 | 0 | 1 | 1 | B (11) |

|   |   |   |   |        |
|---|---|---|---|--------|
| 1 | 0 | 0 | 1 | A (10) |
| 1 | 0 | 1 | 1 | B (11) |
| 1 | 1 | 0 | 0 | C (12) |
| 1 | 1 | 0 | 1 | D (13) |
| 1 | 1 | 1 | 0 | E (14) |
| 1 | 1 | 1 | 1 | F (15) |

## Binary to Hexadecimal Conversion

For the integer part:

- Scan the binary number from right to left.
- Translate each group of four bits into the corresponding hexadecimal digit.
  - Add leading zeros if necessary.

For the fractional part:

- Scan the binary number from left to right.
- Translate each group of four bits into the corresponding hexadecimal digit.
  - Add trailing zeros if necessary.

For example:-

$$\overbrace{10101110101.01101110}^{\text{2 e d b 7}}_2 = (2ed.67)_{16}$$

$$(\underline{0.10000100})_2 = (0.84)_{16}$$

|   |   |   |   |         |
|---|---|---|---|---------|
| 8 | 4 | 2 | 1 | 0 to 15 |
| 1 | 1 | 0 | 1 |         |
| 1 | 1 | 1 | 0 |         |
| 1 | 0 | 0 | 0 |         |
| 0 | 1 | 0 | 1 |         |
| 1 | 0 | 1 | 0 |         |
|   |   | 1 | 1 |         |
|   | 1 | 1 | 0 |         |
|   | 0 | 0 | 1 |         |
|   | 0 | 0 | 1 |         |

## Hexadecimal to binary Conversion

Translate every hexadecimal digit into its 4 bit binary equivalent.

$$(3A5)_{16} = (1110100101)_2$$

$$(12.3D)_{16} = (10010.0011101)_2$$

## Octal Number System

A compact way of representing binary numbers

8 different symbols (radix or base = 8)

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 0 | → | 0 |
| 0 | 0 | 1 | → | 1 |
| 0 | 1 | 0 | → | 2 |
| 0 | 1 | 1 | → | 3 |
| 1 | 0 | 0 | → | 4 |
| 1 | 0 | 1 | → | 5 |
| 1 | 1 | 0 | → | 6 |
| 1 | 1 | 1 | → | 7 |

## Binary to Octal Conversion

For the integer part:

- Scan the binary number from right to left
- Translate each group of three bits into a corresponding octal digit
  - Add leading zeros if necessary

For the fractional part:

- Scan the binary number from left to right
- Translate each group of three bits into the corresponding octal digit
  - Add trailing zeros if necessary

For example:

$$(1010101)_2 = (525)_8$$

$$(0.01010110)_2 = (0.256)_8$$

$$(100101.011010)_2 = (15.32)_8$$

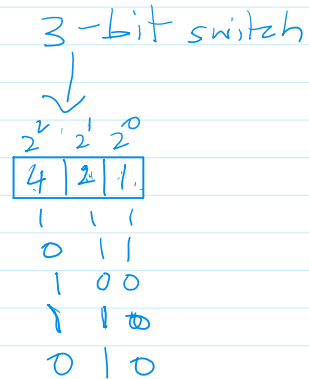
bit → switch  $\begin{cases} \text{ON} \rightarrow 1 \\ \text{OFF} \rightarrow 0 \end{cases}$

### Octal to binary Conversion

Translate every octal digit into its 3 bit binary equivalent.

$$(437)_8 = (100\ 011\ 111)_2$$

$$(53.26)_8 = (101011.010110)_2$$



### Decimal to octal

$$(983)_{10} = (?)_8$$

|   |     |      |
|---|-----|------|
| 8 | 983 |      |
| 8 | 122 | -7 ↑ |
| 8 | 15  | -2   |
| 8 | 1   | -7   |
|   | 0   | -1   |

$$(983)_{10} = (1727)_8$$

$$(8)_{10} = (?)_8$$

|   |   |      |
|---|---|------|
| 8 | 8 |      |
| 8 | 1 | -0 ↑ |
|   | 0 | -1   |

$$(8)_{10} = (10)_8$$

$$1/2048 = 0.0005$$

$$1/64 = 0.0156$$

$$(201.53)_{10} = (?)_8$$

For integer part-

|   |     |     |
|---|-----|-----|
| 8 | 201 |     |
| 8 | 25  | 1 ↑ |
|   | 1   |     |

|   |        |
|---|--------|
|   | 53 × 8 |
| 4 | 24 × 8 |
| 1 | 9 × 8  |
| 7 | 0 × 8  |

$$(311.4172)_{10} = (?)_8$$

$$= 3 \times 8^2 + 1 \times 8^1 + 1 \times 8^0 + 4 \times 8^{-1} + 1 \times 8^{-2} + 7 \times 8^{-3} + 2 \times 8^{-4}$$

$$= 192 + 8 + 1 + \frac{4}{8} + \frac{1}{64} + \frac{7}{512} + \frac{2}{2048}$$

$$\begin{array}{r|l} 8 & 201 \\ \hline 8 & 25 \\ \hline 8 & 3 \\ \hline & 0 \end{array} \quad \begin{array}{l} 1 \uparrow \\ 1 \uparrow \\ 3 \end{array}$$

$$(201.53)_{10} \approx (323.4172)_8$$

$$\begin{array}{r|l} 4 & 24 \times 8 \\ \hline 1 & 92 \times 8 \\ \hline 7 & 36 \times 8 \\ \hline 2 & 88 \end{array}$$

$$\begin{array}{r} 82 \quad 64 \quad 512 \quad 4096 \\ \hline 201 + .5 + 0.0156 + 0.0137 + 0.0005 \\ \hline 201.5298 \end{array}$$

$$0.0156 + 0.0137 + 0.0005 + 0.5 = 0.5298$$

Octal to decimal

### Hexadecimal to decimal

$$(ABC)_{16} = (?)_{10}$$

$$101010111000$$

$$\begin{array}{|c|c|c|c|} \hline 8 & 4 & 2 & 1 \\ \hline 1 & 1 & 0 & 0 \\ \hline 1 & 0 & 1 & 1 \\ \hline 1 & 0 & 1 & 0 \\ \hline \end{array}$$

Hex to bin  $\rightarrow$  dec

$$\begin{array}{|c|c|c|c|c|c|c|c|c|c|c|c|} \hline 2048 & 1024 & 512 & 256 & 128 & 64 & 32 & 16 & 8 & 4 & 2 & 1 \\ \hline 1 & 0 & 1 & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ \hline \end{array}$$

$$2048 + 512 + 128 + 32 + 16 + 8 + 4 = 2748$$

$$2048 + 512 + 128 + 32 + 16 + 8 + 4 = 2748$$

### Unsigned binary Numbers

A n-bit binary number

$$B = b_{n-1} b_{n-2} b_{n-3} \dots b_2 b_1 b_0$$

$2^n$  distinct combinations are possible; 0 to  $2^n - 1$

For example, for  $n=3$ , there are 8 distinct combination.

$$000, 001, 011, 100, 101, 110, 111$$

Range of numbers that can be represented.

$$\begin{array}{ll} n=8 & 0 \text{ to } 2^8 - 1 (255) \\ n=16 & 0 \text{ to } 2^{16} - 1 (65535) \\ n=32 & 0 \text{ to } 2^{32} - 1 (4294967295) \end{array}$$

$$n=8$$

$$\begin{array}{|c|c|c|c|c|c|c|c|} \hline 128 & 64 & 32 & 16 & 8 & 4 & 2 & 1 \\ \hline 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ \hline 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ \hline \end{array}$$

$$\rightarrow 0$$

$$\rightarrow 255$$

$$\text{unsigned integer} = \boxed{0 \text{ to } 2^n - 1} \quad \because n = \text{number of bits}$$

### Signed Integer representation

Many of the numerical data items that are used in programs are signed (positive or negative).

Question:: How to represent sign?

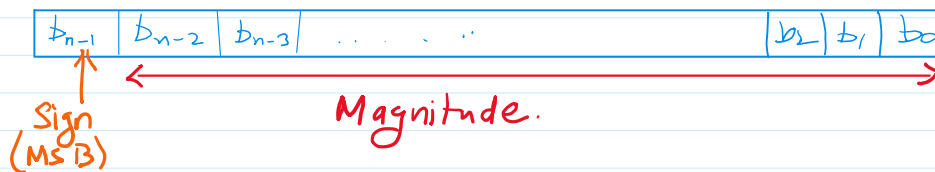
Three possible approaches:

- i. Sign-magnitude representation
- ii. One's complement representation
- iii. Two's complement representation

i. Sign-magnitude representation

For an N - bit representation

- The most significant bit (MSB) indicates sign
  - 0 -> positive
  - 1 -> negative
- The remaining (N - 1) bits represents magnitude



$$n = 8$$

Range of the number that can be represented:

$$\text{Maximum} :: + (2^{n-1} - 1)$$

$$\text{Minimum} :: - (2^{n-1} - 1)$$

$$\text{max} \Rightarrow 2^{8-1} - 1 = 2^7 - 1 = 127$$

$$\text{min} \Rightarrow -1(2^{8-1} - 1) = -1(2^7 - 1) = -127$$

| 7    | 6  | 5  | 4  | 3 | 2 | 1 | 0 |
|------|----|----|----|---|---|---|---|
| -128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 |
| 0    | 1  | 1  | 1  | 1 | 1 | 1 | 1 |
| → 1. | 0  | 0  | 0  | 0 | 0 | 0 | 0 |
| → 0  | 0  | 0  | 0  | 0 | 0 | 0 | 0 |

← +127  
- 0  
+ 0

A problem:

Two different representation of zeros:

+0 -> 0 0000...0

-0 -> 1 0000...0

One's Complement Representation

Basic idea:

- Positive numbers are represented exactly as in sign-magnitude form.
- Negative numbers are represented in 1's complement form.

How to compute 1's complement of a number?

- Complement every bit of the number (1->0 and 0->1)
- MSB will indicate the sign of the number
  - 0 -> positive
  - 1 -> negative

- 0 -> positive
- 1 -> negative

Example ::  $n = 4$

0 0 0 0 -> +0  
 0 0 0 1 -> +1  
 0 0 1 0 -> +2  
 0 0 1 1 -> +3  
 0 1 0 0 -> +4  
 0 1 0 1 -> +5  
 0 1 1 0 -> +6  
 0 1 1 1 -> +7

1's complement → why?

MSB →

|   |   |   |   |       |
|---|---|---|---|-------|
| 1 | 0 | 0 | 0 | -> -7 |
| 1 | 0 | 0 | 1 | -> -6 |
| 1 | 0 | 1 | 0 | -> -5 |
| 1 | 0 | 1 | 1 | -> -4 |
| 1 | 1 | 0 | 0 | -> -3 |
| 1 | 1 | 0 | 1 | -> -2 |
| 1 | 1 | 1 | 0 | -> -1 |
| 1 | 1 | 1 | 1 | -> -0 |

0 0 0 0  
 1. 1 1 1 1  
 + 1  
 1 0 0 0

Range of the number that can be represented:

Maximum ::  $+(2^{n-1} - 1)$

Minimum ::  $-(2^{n-1} - 1)$

A problem:

Two different representation of zero:

+0 -> 0 0 0 0 ... 0

-0 -> 1 1 1 1 ... 1

Advantages of 1's complement representation

- Subtraction can be done using addition
- Leads to substantial saving in circuitry

Subtraction using Addition:: 1's Complement

How to compute  $A - B$ ?

- Compute the 1's complement of B(say, B1)
- Compute  $R = A + B1$
- If the carry obtained after addition is '1'
  - Add the carry back to R (called end-around carry)
  - That is  $R = R + 1$
  - The result is a positive number
- Else
  - The result is negative and is in 1's complement form.

Algorithm.

Example::  $n = 4$  (bits)

6 - 2 using 1's complement

6 → 0 1 1 0  
 2 → 0 0 1 0  
 2' → 1 1 0 1

6 0 1 1 0  
 2' 1 1 0 1  
 1 0 0 1 1  
 + 1  
 0 1 0 0 (+4)

Example 2 :: 3 - 5

3 → 0011  
5 → 0101  
5' → 1010

0011  
1010  
1101

No carry  
it means result is -ve.  
(-2) 0010 = 2

Example:  $-4 - 3 = -7$

$n = 4$  (bits)

4 in binary = 0100  
4' (complement) = 1011

3 in binary = 0011  
3' (complement) = 1100

1011  
+ 1100  
① 0111 ←  
+1  
1000 ← (-7)

What is 1011 = 11 in decimal, or (15-4)

What is 1100 = 12 in decimal, or (15-3)

1011 = 23 in decimal (15+15-4-3)

1000 ← (-7 in 1's comp)

↑ This is an exceptional case, where the 1's complement algorithm fails.  
It fails when A is -ve and B is positive,

$$-A - (+B) = -A - B = -R$$

Subtract:  $4 - 3$

4 binary 0100

3 binary: 0011  
3' complement: 1100

0100  
1100  
10000  
→ +1  
0001

Subtract:  $-4 + 3 = -1$

4 binary 0100  
4' complement 1011

3 binary: 0011

$$\begin{array}{r} 1011 \\ + 0011 \\ \hline 1110 \end{array}$$

$$\begin{array}{r} 10 \\ + 1 \\ \hline 11 \end{array}$$

4 bit

-8 | 4 | 2 | 1

Overflow? Example: 5+6

-7 + 7

$$5 \text{ binary} = 0101$$

$$6 \text{ binary} = 0110$$

$$\begin{array}{r} 0101 \\ 0110 \\ \hline 1011 \end{array} \rightarrow (-ve) \text{ in 1's complement.}$$

Overflows are handled separately

$$n = 8$$

|      |    |    |    |   |   |   |   |
|------|----|----|----|---|---|---|---|
| 7    | 6  | 5  | 4  | 3 | 2 | 1 | 0 |
| -128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 |

$$5 \text{ binary} = 0000 \ 0101$$

$$6 \text{ binary} = + 0000 \ 0110$$

$$0000 \ 1011 \rightarrow +11$$

To handle overflows we need increase number of bits.  
Therefore,  $N = n \times 2$  will be applied. [ $\because n$  is current number of bits]

How to handle the problem of having two 0's?

## Two's Complement Representation

Basic idea:

- Positive numbers are represented exactly as in sign-magnitude form.
- Negative numbers are represented in 2's complement form.

How to compute 2's complement of a number?

- Complement every bit of the number (1  $\rightarrow$  0 and 0  $\rightarrow$  1), and then add one to the resulting number.

- MSB will indicate the sign of the number
  - 0  $\rightarrow$  positive
  - 1  $\rightarrow$  negative

$$\begin{array}{r} \text{sign-bit} \\ -6 \rightarrow 0110 \\ 1001 \leftarrow 1's \text{ compl.} \\ +1 \\ \hline 1010 \leftarrow 2's \text{ compl.} \end{array}$$

Two's complement

Range of numbers that can be represented:

$$\text{Maximum} :: +(2^{n-1} - 1)$$

$$\text{Minimum} :: -(2^{n-1})$$

Advantage:

- A unique representation of zero
- Subtraction can be done using addition
- Leads to substantial saving in circuitry

Almost all computers today use the two's complement representation for storing negative numbers.

Example:  $n = 4$

Subtract  $(6 - 3)$  using 2's complement

Binary of 6 = 0 1 1 0

Binary of 3 = 0 0 1 1

3's complement = 1 1 0 0

The 2's complement of 3 is = 1 1 0 0 + 1 = 1 1 0 1

Add the binary of 6 with 2's complement of 3:

0 1 1 0  
1 1 0 1

$\swarrow$  0 0 1 1 (+3).

Subtract  $(3 - 6)$  using 2's complement

Binary of 3 is = 0 0 1 1

Binary of 6 is = 0 1 1 0

6 one's complement = 1 0 0 1

6 two's complement = 1 0 0 1 + 1 = 1 0 1 0

0 0 1 1  
1 0 1 0  
-----  
1 1 0 1 (no carry) -ve number

$\downarrow$   
(-3)

$\rightarrow$  complement

0 0 1 0  
+ 1  
-----  
0 0 1 1 = 3

Subtract  $(-3 - 6)$  using 2's complement

Binary of 3 = 0 0 1 1

Two's complement = 1 1 0 0 + 1 = 1 1 0 1

Binary of 6 = 0 1 1 0

Two's complement = 1 0 0 1 + 1 = 1 0 1 0

Add:

1 1 0 1  
1 0 1 0  
-----  
1 0 1 1

$\swarrow$  2's complement of 3

$$\begin{array}{r} \times 0111 \\ (-0111) \end{array} \rightarrow \text{2's complement form.}$$

$$1000 + 1 = 1001 = \textcircled{9}$$

### Multiplication of binary number

$$(101101 * 101) = ?$$

$$\begin{array}{r} 101101 \\ \times 101 \\ \hline 101101 \\ 000000 \\ 101101 \\ \hline 11100001 \end{array}$$

$$\begin{array}{l} 0+0=0 \\ 0+1=1 \\ 1+0=1 \\ 1+1=0 \text{ (1 carry)} \\ 10+1=11 \\ 11+1=00 \text{ (1 carry)} \end{array}$$

### Multiplication of octal number

$$(726_8 \times 25_8) = ?$$

$$\begin{array}{r} 726 \\ \times 25 \\ \hline 4456 \\ 1654 \times \\ \hline 23216_8 \end{array}$$

|                                | 8 | 2 |
|--------------------------------|---|---|
| 30 <sub>10</sub> $\Rightarrow$ | 3 | 6 |
| 12 $\Rightarrow$               | 1 | 4 |
| 13 $\Rightarrow$               | 1 | 5 |
| 36 $\Rightarrow$               | 4 | 4 |

The rule is:

1. Do operation
2. Convert the answer into given number system.

### Multiplication of hexadecimal Number system.

$$FACE_{16} \times BAD_{16}$$

$$\begin{array}{r} FACE \\ \times BAD \\ \hline 2CBC76 \\ 19CC0CX \\ AC6E5XX \\ \hline B706236_{16} \end{array}$$

|               |    |   |
|---------------|----|---|
| 17x13+0=182   | 11 | 6 |
| 12x13+11=167  | 10 | 7 |
| 10x13+10=140  | 8  | C |
| 15x13+8=203   | 12 | B |
| 10x14+0=140   | 8  | C |
| → 10x12+8=128 | 8  | 0 |
| 10x10+8=108   | 6  | C |
| 15x10+6=156   | 9  | C |
| 15x11+0=165   | 10 | 5 |
| 12x11+10=142  | 8  | E |
| 10x11+8=118   | 7  | 6 |
| 15x11+7=172   | 10 | C |

$$\begin{array}{r} 16 \\ \times 8 \\ \hline 128 \end{array}$$

$$10 \times 11 + 8 = 118$$

$$15 \times 11 + 7 = 172$$

$$\begin{array}{r} 10 \ 6 \\ 10 \ 6 \end{array}$$

Division of binary.

Q      R

$$(101011)_2 \div (1110)_2 = ?$$

$$\begin{array}{r} 1110 \overline{) 101011} \\ \underline{-1110} \phantom{00} \\ 001111 \\ \underline{-1110} \phantom{00} \\ 00011 \\ \underline{-00} \\ 11 \end{array} \quad \begin{array}{l} (110 \leftarrow \text{quotient} \\ 11 \leftarrow \text{remainder} \end{array}$$

$$\begin{array}{r} 10 < 14 \\ 15 > 14 \\ 3 < 14 \end{array}$$

Quotient in decimals.

$$\begin{array}{r} 1110 \overline{) 101011} \\ \underline{-1110} \phantom{00} \\ 001111 \\ \underline{-1110} \phantom{00} \\ 00011 \\ \underline{-00} \\ 110 \\ \underline{-000} \\ 1100 \\ \underline{-000} \\ 100 \\ \underline{-1110} \\ 1010 \end{array}$$

Ans: quotient  $\approx 110.001$ . (approx 3 decimal places)

Octal division.

$$75_8 \div 23_8$$

$$\begin{array}{r} 23 \overline{) 751} \\ \underline{71} \phantom{00} \\ 419 \\ \underline{323} \\ 16 \end{array} \quad \begin{array}{l} (31 \leftarrow \text{quotient} \\ 16 \leftarrow \text{remainder} \end{array}$$

$$\begin{array}{r} 23 \\ \times 3 \\ \hline 71 \end{array}$$

Octal division in decimals

## Octal division in decimals

$$\begin{array}{r} 23 \overline{) 751} \quad (31.62 \dots) \\ \underline{71} \phantom{00} \\ 419 \\ \underline{23} \phantom{00} \\ 1808 \\ \underline{152} \phantom{00} \\ 00508 \\ \underline{-46} \phantom{00} \\ 12 \end{array}$$

$$751 \div 23 \approx 31.62$$

$$\begin{array}{r} 1 \\ 23 \\ \times 6 \\ \hline 152 \end{array}$$

$$\begin{array}{r} 23 \\ \times 2 \\ \hline 46 \end{array}$$

$$\begin{array}{r} 5 \\ AD \\ \times 7 \\ \hline \rightarrow 4BB \end{array}$$

$$\begin{array}{l} 91 \rightarrow 5, B \\ 75 \rightarrow 4, B \end{array}$$

## Hexadecimal division

$$FACE_{16} \div AD_{16}$$

$$\begin{array}{r} AD \overline{) FACE} \quad (173 \leftarrow \text{quotient}) \\ \underline{-AD} \phantom{000} \\ 4DC \\ \underline{4BB} \phantom{00} \\ 021E \\ \underline{207} \phantom{00} \\ 17 \leftarrow \text{rem} \end{array}$$

$$\begin{array}{r} 1 \\ AD \\ \times 2 \\ \hline 15A \\ 3 \\ AD \\ \times 4 \\ \hline 2B4 \end{array}$$

$$\begin{array}{r} 26 \rightarrow 1A \\ 39 \rightarrow 27 \\ 20 \\ 52 \rightarrow 34 \\ 43 \rightarrow 2B \\ 78 \rightarrow 4E \\ 740 \\ 104 \rightarrow 68 \\ 86 \rightarrow 56 \end{array}$$

$$\begin{array}{r} 4 \\ AD \\ \times 6 \\ \hline 40E \\ 6 \\ AD \\ \times 8 \\ \hline 568 \end{array}$$

## Hexadecimal division in decimal points

$$FACE_{16} \div AD_{16}$$

$$\begin{array}{r} AD \overline{) FACE} \quad (173.219) \\ \underline{-AD} \phantom{000} \\ 4DC \\ \underline{4BB} \phantom{00} \\ 021E \\ \underline{207} \phantom{00} \\ 1016 \\ \underline{15A} \phantom{00} \\ 45016 \\ \underline{AD} \phantom{00} \\ 9A25016 \\ \underline{6B5} \phantom{00} \\ 37B \end{array}$$

$$\begin{array}{r} 8 \\ AD \\ \times A \\ \hline 6C2 \end{array}$$

$$\begin{array}{r} 17 \\ AD \\ \times 9 \\ \hline 6B5 \\ 8 \\ AD \\ \times B \\ \hline 76F \end{array}$$

$$\begin{array}{l} 117 \rightarrow 7, 5 \\ 107 \rightarrow 6, B \\ 130 \rightarrow 8, 2 \\ 108 \rightarrow 6, C \\ 143 \rightarrow 8, F \\ 118 \rightarrow 7, 6 \end{array}$$

$$FACE \div AD \approx 173.219$$

$$\begin{array}{r} 9 \\ AD \\ \times C \\ \hline 81C \\ 10 \\ AD \\ \times D \\ \hline 8C9 \end{array}$$

$$\begin{array}{r} AD \\ \times E \\ \hline \end{array}$$

$$\begin{array}{l} 156 \rightarrow 9, C \\ 129 \rightarrow 8, 1 \\ 169 \rightarrow 10, 9 \\ 140 \rightarrow 8, C \end{array}$$

On UT  $\rightarrow$  Do the question in descending order of marks.

9's complement

$$105_{10} - 96_{10}$$

$$105 - 096$$

-ve number, we need to find  
9's complement

$$\begin{array}{r} 999 \\ - 096 \\ \hline 903 \end{array} \leftarrow 9 \text{ complement}$$

$$\begin{array}{r} 105 \\ 903 \\ \hline 1008 \\ +1 \\ \hline 009 \end{array} \leftarrow \underline{\text{Ans.}}$$

$$905_{10} - 23_{10}$$

$$\begin{array}{r} 999 \\ 023 \\ \hline 976 \end{array} \leftarrow 4\text{'s complement}$$

$$\begin{array}{r} 905 \\ 976 \\ \hline 1 \text{ } 881 \\ \quad \rightarrow +1 \\ \hline 882 \end{array} \leftarrow \text{Ans}$$

## 10's complement

$$105_{10} - 96_{10}$$

096

9's complement of '096'  $\rightarrow$

|       |
|-------|
| 999   |
| 096   |
| <hr/> |
| 903   |
| +1    |
| <hr/> |
| 904   |

10 complement  $\rightarrow$

$$\begin{array}{r} 999 \\ 096 \\ \hline 903 \\ +1 \\ \hline \rightarrow 904 \end{array}$$

1's complement

$$\begin{array}{r} 105 \\ 904 \\ \hline \cancel{1}009 \end{array}$$

on 10's complement we discard the carry.

### Conversion:

(conversion:

(i)  $(1011011101)_2 = ( ? )_{10} = 263.625$

$$\begin{array}{r} .625 \\ 8 \overline{) 50} \\ \underline{48} \phantom{00} \\ 20 \\ \underline{16} \phantom{00} \\ 40 \end{array}$$

$$(2) \quad (62.57)_{10} = (\quad ? \quad)_2 \approx$$

(3)  $ABCD_{16} = ( \quad ? \quad )_{10} \quad 43981$

(4)  $67098_{10} = ( \quad ? \quad )_{16}$

$$\begin{array}{r} 1.57 \times 2 \\ 1.14 \times 2 \\ 0.28 \times 2 \\ 0.56 \times 2 \\ 1.12 \end{array}$$

$$(4) \quad 67098_{10} = ( \quad ? \quad )_{16}$$

$$\begin{array}{r} 0 \quad \sim \quad - \\ 1 \cdot 12 \end{array}$$

$$(5) \quad (89.25)_{10} = ( \quad ? \quad )_8 \quad \checkmark$$

$$\begin{array}{r} \cdot 25 \times 8 \\ 2 \cdot 00 \end{array}$$

$$(6) \quad (25.23)_8 = ( \quad ? \quad )_{10}$$

$$(7) \quad (6A.2B)_{16} = ( \quad )_2 \quad \mathbf{1101010.00101011}$$

$$(8) \quad \text{Subtract } 1100101_2 - 11101_2 \text{ using 2's complement}$$

$$(9) \quad \text{Subtract } 11001_2 - 11101_2 \text{ using 2's complement}$$

$$\begin{array}{r} 0011101 \\ 1100010 + 1 \\ \hline 1100011 \end{array}$$

$$\begin{array}{r} 00010 + 1 \\ = 00011 \\ \underline{11001} \\ 11100 \end{array}$$

Subtract:  $(1111 - 1000001)$  solve using 2's complement.

$$\begin{array}{r} 01060001 \\ \downarrow \\ 10111110 \\ \hline 00001111 \\ + 10111110 \\ \hline 11001101 \\ \hline -128 + 64 + 8 + 4 + 1 \end{array}$$

1. equalize number of digits by adding '0' on the left side.
2. Do the 2's complement of '-ve value.
3. Add the result.

← no carry means (-ve) value

$$-128 + 64 + 8 + 4 + 1 = -51 + 1 = -50$$